Course Outline: Data Types and Basic Operators in TypeScript

Title: Data Types and Basic Operators in TypeScript **Prerequisite:** Introduction to Programming Logic (Week 2, Day 6) **Target Audience:** Beginner developers learning TypeScript fundamentals for the first time **Total Lessons:** 18 **Estimated Total Length:** ~9,000 words **Format:** Practical (44% hands-on)

Course Description

This course introduces students to the fundamental building blocks of TypeScript programming: data types and operators. Students will learn to work with primitive data types (numbers, booleans, strings, symbols, null, and undefined) and understand how to manipulate data using arithmetic, assignment, comparison, and logical operators.

The course emphasizes practical understanding over memorization, teaching students to identify data types by their composition and use cases rather than syntax alone. Through hands-on exercises, students will develop the ability to convert pseudocode solutions into TypeScript syntax while building confidence in working with different data types and operations.

This foundation is critical for all future TypeScript work, as every program relies on storing data in variables and manipulating that data through operations. Students will learn to write clean, semantic code following TypeScript conventions while avoiding common beginner mistakes.

Course Philosophy

This course emphasizes:

- **Investigation over memorization**: Learning to look up syntax when needed rather than memorizing every detail
- **Type awareness**: Understanding data types by their composition and behavior, not just their names
- **Practical experimentation**: Writing code (not copying) to develop muscle memory and understanding
- **Semantic clarity**: Using meaningful variable names and strict equality for readable, maintainable code
- **Disciplined fundamentals**: Writing one instruction per line with proper indentation and semicolons

---

Course Statistics Summary

Section	Lessons	Estimated Words
00 - Welcome to TypeScript Fundamentals	1	~450
01 - Primitive Data Types	4	~2,150
02 - Arithmetic and Assignment Operators	3	~1,650
03 - Comparison and Logical Operators	4	~2,250
04 - Type Identification and Conversion	3	~1,650
05 - Practice and Assessment	2	~1,450
06 - Conclusion	1	~450
Total	**18**	**~9,000**

Section 00: Welcome to TypeScript Fundamentals

00.0 Introduction: Understanding Data and Operations 📖 (~450 words)

Learning Objectives:

- Understand what data types and operators are and why they matter
- Recognize the relationship between data types and real-world information
- Identify the role of operators in manipulating data
- Connect previous programming logic knowledge to TypeScript syntax

Content Outline:

- What are data types? (Categories of information computers can store)
- What are operators? (Tools for manipulating and comparing data)
- Why TypeScript cares about types (Type safety prevents errors)
- The relationship between pseudocode and TypeScript syntax
- How this course builds on programming logic foundations

Transitions: This introduction sets the stage for understanding TypeScript's type system. The next lesson will explore the primitive data types that form the foundation of all TypeScript programs.

Key Principles:

- Every piece of data has a type that determines what you can do with it
- Operators allow you to transform, compare, and combine data
- TypeScript's type system helps catch errors before they happen
- Understanding data types by composition (what they look like) is more important than memorizing definitions

Examples:

- Real-world analogy: Data types are like containers (number = calculator, string = notepad, boolean = light switch)
- Operators in daily life: Adding prices (arithmetic), comparing ages (comparison), deciding eligibility (logical)

Section 01: Primitive Data Types

01.0 Understanding Primitive Data Types 📖 (~500 words)

Learning Objectives:

- Define what primitive data types are in TypeScript
- Identify the six primitive types: number, boolean, string, symbol, null, undefined
- Understand why TypeScript distinguishes between different primitive types
- Recognize primitive types by their composition and appearance

Content Outline:

- Definition: Primitive types are the simplest, most basic data types
- Overview of the six primitive types and their purposes
- How TypeScript uses types to prevent errors
- The difference between primitive types and complex types (preview)
- Type inference: How TypeScript automatically detects types

Transitions: Now that you understand what primitive types are, the next lessons will explore each type in detail, starting with numbers and booleans.

Key Principles:

- Primitive types are the building blocks of all TypeScript programs
- Each type has specific use cases and behaviors
- TypeScript infers types automatically, but you can specify them explicitly

- Identifying types by composition (how they look) is essential

Examples:

- `42` is a number (no quotes, can do math)
- `true` is a boolean (only two possible values: true or false)
- `"Hello"` is a string (surrounded by quotes)
- `undefined` means "not set yet"
- `null` means "intentionally empty"

01.1 Working with Numbers and Booleans 💻 (~550 words)

Learning Objectives:

- Declare and initialize variables with number and boolean types
- Understand the use cases for numbers and booleans
- Recognize type inference with numbers and booleans
- Practice writing clean, semantic variable declarations

Content Outline:

- Number type: Integers and decimals (TypeScript doesn't distinguish)
- Boolean type: true and false values only
- Declaring variables with const and let
- Type inference examples with numbers and booleans
- Explicit type annotations (optional but clear)
- Common use cases for each type

Transitions: With numbers and booleans mastered, you're ready to explore strings and symbols, the types used for text and unique identifiers.

Key Principles:

- Use const by default, let only when the value will change
- TypeScript infers types automatically from initial values
- Numbers can be positive, negative, or decimals
- Booleans represent yes/no, on/off, true/false decisions

Examples:

```
 const age: number = 25;
let score = 0; // Type inferred as number
const isActive: boolean = true;
let hasAccess = false; // Type inferred as boolean
```

Hands-On Exercise: Create variables for a student profile:

1. Declare a constant for student age (number)
2. Declare a variable for current score (number, can change)
3. Declare a constant for enrollment status (boolean)
4. Declare a variable for course completion (boolean, can change)

Practice Scenario: You're building a game character system. Create variables for:

- Health points (number, can change)
- Maximum health (number, constant)
- Is alive (boolean)
- Power level (number)

Success Criteria:

- Variables use appropriate types (number or boolean)
- Const is used for values that won't change
- Let is used for values that will change
- Variable names are semantic and meaningful

01.2 Strings and Symbols Explained 📖 (~550 words)

Learning Objectives:

- Understand the string type and its use cases
- Learn different ways to create strings (single, double quotes, backticks)
- Understand the symbol type and its unique properties
- Recognize when to use strings vs symbols

Content Outline:

- String type: Text data surrounded by quotes
- Three ways to create strings: 'single', "double", `template`
- Template literals and string interpolation (preview)
- String concatenation basics
- Symbol type: Unique identifiers that are always different
- Use cases for symbols (advanced, but important to know they exist)
- Why symbols are primitive despite being unique

Transitions: You've now learned about the "positive" primitive types (types that hold values). Next, you'll explore null and undefined—the types that represent absence of data.

Key Principles:

- Strings hold text data and are always surrounded by quotes
- Use backticks for template literals when combining strings with variables
- Symbols are unique identifiers, always different from each other
- Each string method returns a new string (strings are immutable)

Examples:

```
 const firstName = "Maria";
const lastName = 'Rodriguez';
const greeting = `Hello, ${firstName}!`; // Template literal

const id = Symbol("user-id");
const anotherId = Symbol("user-id");
// id !== anotherId (symbols are always unique)
```

01.3 Null, Undefined, and Type Safety 💻 (~550 words)

Learning Objectives:

- Understand the difference between null and undefined
- Learn when to use null vs undefined
- Practice checking for null and undefined values
- Understand TypeScript's type safety with these values

Content Outline:

- Undefined: Variable declared but not assigned a value
- Null: Intentionally empty or "no value"
- The philosophical difference: undefined = "not set yet", null = "explicitly nothing"
- Checking for null and undefined
- TypeScript's strict null checking feature
- Common mistakes with null and undefined

Transitions: Now that you understand all six primitive types, you're ready to learn how to manipulate these values using operators, starting with arithmetic and assignment.

Key Principles:

- Undefined means a variable exists but has no value assigned
- Null means you intentionally set something to "empty"
- Always check for null/undefined before using values
- TypeScript helps prevent errors by tracking which variables might be null/undefined

Examples:

```
 let userName: string | undefined;
console.log(userName); // undefined (declared but not assigned)

let result: number | null = null;
// Later... result = 42;

// Checking for null/undefined
if (userName !== undefined) {
    console.log(userName.toUpperCase());
}
```

Hands-On Exercise: Practice working with null and undefined:

1. Declare a variable without initializing it—observe undefined
2. Declare a variable and explicitly set it to null
3. Write a function that returns null when data isn't found
4. Check for null/undefined before using a value

Practice Scenario: You're building a user profile system:

- Create a variable for optional phone number (might be undefined)
- Create a variable for deleted account (should be null when deleted)
- Write checks before displaying these values

Success Criteria:

- Correctly distinguish between null (intentional) and undefined (not set)
- Check for null/undefined before using values
- Use appropriate types (string | null or string | undefined)
- Understand when each is appropriate

Section 02: Arithmetic and Assignment Operators

02.0 Arithmetic Operators in TypeScript 📖 (~500 words)

Learning Objectives:

- Learn the five basic arithmetic operators: +, -, *, /, %
- Understand operator precedence (order of operations)

- Recognize how TypeScript handles number operations
- Identify common use cases for each arithmetic operator

Content Outline:

- Addition (+): Combining numbers
- Subtraction (-): Finding differences
- Multiplication (*): Scaling values
- Division (/): Splitting into parts
- Modulo (%): Finding remainders (very useful for cycles, even/odd)
- Operator precedence: PEMDAS applies in programming
- Grouping with parentheses for clarity
- Integer vs decimal results in division

Transitions: Understanding arithmetic operators is essential, but you'll often want to update existing values. The next lesson covers assignment operators that make this efficient.

Key Principles:

- Arithmetic operators work just like math: +, -, *, /, %
- Order of operations matters (multiplication before addition, etc.)
- Use parentheses to make your intention clear
- The modulo operator (%) is incredibly useful for cycles and patterns

Examples:

```
 const sum = 10 + 5; // 15
const difference = 10 - 5; // 5
const product = 10 * 5; // 50
const quotient = 10 / 5; // 2
const remainder = 10 % 3; // 1 (useful for even/odd checks)

// Precedence
const result = 2 + 3 * 4; // 14 (not 20)
const clarified = (2 + 3) * 4; // 20
```

02.1 Assignment Operators and Shortcuts 💻 (~575 words)

Learning Objectives:

- Master the basic assignment operator (=)
- Learn compound assignment operators (+=, -=, *=, /=, %=)
- Understand increment (++) and decrement (--) operators
- Practice writing efficient update operations

Content Outline:

- Basic assignment (=): Storing values in variables
- Compound assignment shortcuts: +=, -=, *=, /=, %=
- When compound operators improve readability
- Increment (++) and decrement (--) operators
- Prefix vs postfix (++x vs x++): Brief mention, prefer simple form
- Common patterns with assignment operators

Transitions: With arithmetic and assignment mastered, you're ready to compare values and make decisions—the topic of the next section on comparison and logical operators.

Key Principles:

- Assignment (=) stores a value in a variable
- Compound operators (+=, -=, etc.) are shortcuts for common patterns
- Use compound operators to make your intention clear
- Increment/decrement operators are convenient but can be confusing

Examples:

```
 let score = 0;
score = score + 10; // Long form
score += 10; // Shortcut (same result)

let lives = 3;
lives -= 1; // Remove one life
lives--; // Even shorter (same as lives -= 1)

let multiplier = 2;
multiplier *= 3; // multiplier = multiplier * 3
```

Hands-On Exercise: Practice with assignment operators:

1. Create a counter variable starting at 0
2. Increment it by 5 using +=
3. Double it using *=
4. Decrement it by 1 using --
5. Find remainder when divided by 3 using %=

Practice Scenario: You're building a shopping cart:

- Create a total price variable
- Add item prices using +=
- Apply a discount using -=
- Apply tax by multiplying total using *=

Success Criteria:

- Use appropriate compound operators (+=, -=, *=, /=)
- Understand when shortcuts improve readability
- Variables update correctly with each operation
- Code is clean and easy to understand

02.2 Mathematical Operations Practice 💻 (~575 words)

Learning Objectives:

- Combine multiple arithmetic operations in expressions
- Apply operator precedence correctly
- Solve practical calculation problems
- Write clean, readable mathematical expressions

Content Outline:

- Building complex expressions from simple operators
- Using parentheses for clarity and correctness
- Real-world calculation examples (prices, measurements, time)
- Common calculation patterns in programming
- Debugging mathematical expressions
- console.log() for tracking calculation steps

Transitions: You've now mastered working with numbers through arithmetic and assignment. The next section teaches you to compare values and combine conditions—essential skills for making decisions in your programs.

Key Principles:

- Break complex calculations into smaller steps
- Use descriptive variable names for intermediate results
- Test calculations with console.log() to verify correctness
- Parentheses improve both correctness and readability

Examples:

```
 // Calculate final price with tax and discount
const originalPrice = 100;
const discountPercent = 20;
const taxRate = 0.08;

const discountAmount = originalPrice * (discountPercent / 100);
const priceAfterDiscount = originalPrice - discountAmount;
const finalPrice = priceAfterDiscount * (1 + taxRate);

console.log("Final price:", finalPrice); // 86.4
```

Hands-On Exercise: Create a tip calculator:

1. Define bill amount (e.g., 50)
2. Define tip percentage (e.g., 18)
3. Calculate tip amount
4. Calculate total with tip
5. If splitting bill: calculate per-person amount (4 people)
6. Use console.log() to show each step

Practice Scenario: Build a time converter:

- Convert minutes to hours and remaining minutes
- Example: 135 minutes = 2 hours and 15 minutes
- Hint: Use division (/) and modulo (%)

Success Criteria:

- Complex calculations broken into clear steps
- Parentheses used appropriately
- Variable names describe what they contain
- Results are mathematically correct
- Code is readable and well-organized

Section 03: Comparison and Logical Operators

03.0 Comparison Operators: Making Decisions 📖 (~500 words)

Learning Objectives:

- Learn the six comparison operators: ==, ===, !=, !==, <, >, <=, >=
- Understand the difference between == and === (loose vs strict equality)
- Use comparison operators to create boolean expressions
- Recognize when to use each comparison operator

Content Outline:

- What comparison operators do: Return true or false
- Equality operators: == (loose) vs === (strict)
- Inequality operators: != (loose) vs !== (strict)
- Relational operators: <, >, <=, >=
- Why you should always use === and !== (strict equality)
- Type coercion dangers with == and !=
- Chaining comparisons (what NOT to do)

Transitions: Comparison operators let you ask questions about single values. The next lesson teaches logical operators, which let you combine multiple questions into complex conditions.

Key Principles:

- Comparison operators always return boolean values (true or false)
- Always use strict equality (===) and strict inequality (!==)
- Loose equality (==) performs type coercion and causes unexpected behavior
- Comparisons create conditions for if statements and loops

Examples:

```
 const age = 25;
console.log(age === 25); // true (strict equality)
console.log(age === "25"); // false (different types)
console.log(age == "25"); // true (type coercion - avoid this!)

console.log(age > 18); // true
console.log(age <= 25); // true
console.log(age !== 30); // true
```

03.1 Logical Operators: AND, OR, NOT 📖 (~550 words)

Learning Objectives:

- Master the three logical operators: && (AND), || (OR), ! (NOT)
- Understand how logical operators combine boolean values
- Learn operator precedence with logical operators
- Recognize common patterns with logical operators

Content Outline:

- AND operator (&&): Both conditions must be true
- OR operator (||): At least one condition must be true
- NOT operator (!): Inverts a boolean value
- Truth tables for each operator
- Combining logical operators
- Operator precedence: NOT, then AND, then OR
- Short-circuit evaluation (preview)
- Common logical patterns

Transitions: Now that you understand both comparison and logical operators individually, the next lesson will show you how to combine them effectively to create powerful conditional logic.

Key Principles:

- AND (&&) requires ALL conditions to be true
- OR (||) requires AT LEAST ONE condition to be true
- NOT (!) flips true to false and false to true
- Precedence: ! before && before ||

Examples:

```
 const age = 25;
const hasLicense = true;

// AND: Both must be true
const canDrive = age >= 18 && hasLicense; // true

// OR: At least one must be true
const isAdult = age >= 18 || age >= 21; // true

// NOT: Inverts the value
const isMinor = !(age >= 18); // false

// Combining operators
const canRentCar = (age >= 25 && hasLicense) || age >= 30;
```

03.2 Combining Operators Effectively 💻 (~600 words)

Learning Objectives:

- Combine comparison and logical operators in complex expressions
- Write readable conditional expressions using parentheses
- Solve real-world decision-making problems with operators
- Debug complex boolean expressions

Content Outline:

- Building complex conditions from simple comparisons
- When to use parentheses for clarity
- Common patterns: range checks, multiple requirements
- De Morgan's Laws (simplifying NOT with AND/OR)
- Breaking complex conditions into intermediate variables
- Using console.log() to debug boolean expressions

Transitions: You now know how to create complex conditions correctly. The next lesson highlights common mistakes developers make with operators so you can avoid them.

Key Principles:

- Use parentheses to make your intention clear
- Break very complex conditions into intermediate boolean variables
- Test conditions with various inputs to ensure correctness
- Simpler conditions are easier to understand and maintain

Examples:

```
 const age = 25;
const income = 50000;
const creditScore = 720;

// Complex condition with clear grouping
const qualifiesForLoan =
    (age >= 21 && age <= 65) &&
    (income >= 30000) &&
    (creditScore >= 650);

// Breaking into intermediate variables
```

```
const isWorkingAge = age >= 21 && age <= 65;
const hasSufficientIncome = income >= 30000;
const hasGoodCredit = creditScore >= 650;
const qualifies = isWorkingAge && hasSufficientIncome && hasGoodCredit;
```

Hands-On Exercise: Create an eligibility checker for a program:

1. User must be between 18 and 35 years old
2. User must have completed high school OR have 2+ years experience
3. User must be a resident OR have a visa
4. Write the complete condition
5. Test with different scenarios

Practice Scenario: Build a discount calculator that applies different discounts:

- 20% discount if: member AND (purchase > 100 OR birthday month)
- 10% discount if: first-time customer OR purchase > 200
- 5% discount if: none of the above but subscribed to newsletter

Success Criteria:

- Complex conditions are correctly structured
- Parentheses clearly show grouping
- All test cases produce correct results
- Code is readable and well-organized
- Boolean expressions are efficient (no redundant checks)

03.3 Common Operator Mistakes 📖 (~600 words)

Learning Objectives:

- Identify common mistakes with comparison operators
- Recognize logical operator pitfalls
- Understand type coercion issues with loose equality
- Learn to avoid "kilometric conditions" (overly complex expressions)

Content Outline:

- Using == instead of === (type coercion problems)
- Confusing = (assignment) with === (comparison)
- Writing conditions that are always true or always false
- "Kilometric conditions": Overly complex, hard-to-read expressions
- Missing parentheses causing incorrect precedence
- Negating complex conditions incorrectly
- Testing conditions: How to verify your logic is correct

Transitions: You've learned to use comparison and logical operators correctly while avoiding common mistakes. The next section explores how TypeScript helps you work with types safely through identification and conversion.

Key Principles:

- Always use === and !==, never == and !=
- Keep conditions simple and readable
- Use intermediate variables for complex logic
- Test your conditions with various inputs

Examples:

Wrong:

```
 // Type coercion surprises
console.log(0 == false); // true (avoid!)
console.log("5" == 5); // true (avoid!)

// Assignment instead of comparison
if (x = 5) { // Assigns 5 to x, always true!
    // This is a bug
}

// Kilometric condition (hard to read)
if (age >= 18 && age <= 65 && income > 30000 || hasVisa && !isStudent || creditScore >= 700 && employed === true
    // Too complex!
}
```

Right:

```
 // Strict equality
console.log(0 === false); // false (correct!)
console.log("5" === 5); // false (correct!)

// Comparison
if (x === 5) {
    // Correct comparison
}

// Readable conditions
const isWorkingAge = age >= 18 && age <= 65;
const hasGoodIncome = income > 30000;
const meetsFinancialCriteria = creditScore >= 700 && employed && debt < 10000;

if (isWorkingAge && (hasGoodIncome || meetsFinancialCriteria)) {
    // Clear and readable
}
```

Key Anti-Patterns:

- **Type coercion**: Using == instead of ===
- **Magic code**: Copying conditions without understanding them
- **Kilometric conditions**: Overly long, complex expressions without breaks
- **Inconsistent syntax**: Mixing styles (sometimes ===, sometimes ==)

What to Do Instead:

- Use === and !== exclusively
- Break complex conditions into named boolean variables
- Test conditions with console.log() and various inputs
- Keep each condition focused on one logical check

Section 04: Type Identification and Conversion

04.0 Identifying Data Types 📖 (~500 words)

Learning Objectives:

- Use the typeof operator to identify data types
- Understand what typeof returns for each primitive type

- Learn to identify types by composition (how they look)
- Recognize type identification use cases

Content Outline:

- The typeof operator: Asking "what type is this?"
- What typeof returns for each primitive type
- Special cases: typeof null returns "object" (JavaScript quirk)
- Why you don't always need typeof (visual identification)
- When typeof is useful (dynamic data, debugging)
- Type guards: Using typeof for conditional logic

Transitions: Identifying types is useful, but sometimes you need to convert data from one type to another. The next lesson covers type conversion and coercion.

Key Principles:

- typeof returns a string describing the type
- You can often identify types visually: quotes = string, true/false = boolean, numbers have no quotes
- typeof is most useful when working with dynamic data or debugging
- Learning to identify types by composition is more important than always using typeof

Examples:

```
 console.log(typeof 42); // "number"
console.log(typeof "hello"); // "string"
console.log(typeof true); // "boolean"
console.log(typeof undefined); // "undefined"
console.log(typeof null); // "object" (quirk!)
console.log(typeof Symbol("id")); // "symbol"

// Practical use in conditions
const value: unknown = getUserInput();
if (typeof value === "number") {
    console.log(value * 2); // Safe to do math
}
```

04.1 Type Conversion and Coercion 💻 (~575 words)

Learning Objectives:

- Understand the difference between conversion (explicit) and coercion (implicit)
- Learn explicit conversion methods: Number(), String(), Boolean()
- Recognize implicit coercion situations and avoid them
- Practice converting between types safely

Content Outline:

- Conversion vs coercion: Explicit vs implicit
- Converting to number: Number(), parseInt(), parseFloat()
- Converting to string: String(), .toString(), template literals
- Converting to boolean: Boolean() and truthy/falsy values
- Implicit coercion: When JavaScript/TypeScript converts automatically (often problematic)
- Why explicit conversion is safer
- Handling conversion errors (NaN, invalid inputs)

Transitions: Now that you understand type conversion, the final lesson in this section will give you hands-on practice with the typeof operator and type conversions.

Key Principles:

- Explicit conversion (you do it) is safer than implicit coercion (happens automatically)
- Always validate conversions—not all strings can become numbers
- NaN (Not a Number) indicates a failed number conversion
- Truthy/falsy: 0, "", null, undefined, NaN are falsy; everything else is truthy

Examples:

```
 // Explicit conversion (good)
const age = Number("25"); // 25
const ageString = String(25); // "25"
const isActive = Boolean(1); // true

// Handling conversion failure
const invalid = Number("abc"); // NaN
console.log(isNaN(invalid)); // true

// Implicit coercion (avoid)
const result = "5" + 3; // "53" (string concatenation, not math)
const sum = Number("5") + 3; // 8 (correct)
```

Hands-On Exercise: Create a type converter utility:

1. Take a string input: "42"
2. Convert to number using Number()
3. Check if conversion succeeded (not NaN)
4. Convert the number back to string
5. Convert to boolean
6. Use console.log() to show each step and result

Practice Scenario: Build a form data processor:

- Receive age as string from form: "25"
- Receive "true" or "false" as string for newsletter subscription
- Convert age to number for validation
- Convert newsletter string to boolean
- Handle invalid inputs gracefully

Success Criteria:

- Explicit conversion methods used (Number, String, Boolean)
- NaN is checked after number conversion
- Invalid inputs are handled gracefully
- No implicit coercion occurs
- Results are correct for all test cases

04.2 Working with typeof Operator 💻 (~575 words)

Learning Objectives:

- Practice using typeof in real-world scenarios
- Build type-checking functions
- Combine typeof with conditional logic
- Debug type-related issues using typeof

Content Outline:

- Building type-checking utility functions
- Using typeof for input validation
- typeof in conditional expressions
- Debugging with typeof to find type mismatches
- Handling unknown or any types safely
- typeof limitations and when to use other approaches

Transitions: You've mastered data types, operators, and type manipulation. The next section provides practice challenges and assessment to solidify your understanding.

Key Principles:

- typeof is a tool for runtime type checking
- Use typeof when working with dynamic or unknown data
- Combine typeof with validation logic for robust code
- TypeScript's compile-time types are usually better than runtime typeof

Examples:

```
 // Type-checking utility
function isNumber(value: unknown): boolean {
    return typeof value === "number" && !isNaN(value);
}

// Input validation
function processAge(input: unknown): number | null {
    if (typeof input === "string") {
        const age = Number(input);
        return isNaN(age) ? null : age;
    }
    if (typeof input === "number") {
        return input;
    }
    return null;
}

// Debugging type issues
const data = fetchData();
console.log("Data type:", typeof data);
console.log("Data value:", data);
```

Hands-On Exercise: Build a safe calculator:

1. Create a function that accepts two unknown parameters
2. Check if both are numbers using typeof
3. If not numbers, try converting from string
4. If conversion fails, return an error message
5. If both are numbers, perform addition
6. Test with: numbers, strings, mixed types, invalid inputs

Practice Scenario: Create a data validator for user registration:

- Validate username (must be string, 3+ characters)
- Validate age (must be number or convertible, 18-120)
- Validate email (must be string, contains @)
- Validate terms accepted (must be boolean or "true"/"false")
- Return object with validation results for each field

Success Criteria:

- typeof used correctly for type checking
- Type conversions attempted when appropriate
- Invalid inputs handled with helpful messages
- Functions work with various input types
- Code is defensive and doesn't crash on bad input

Section 05: Practice and Assessment

05.0 Data Types and Operators Challenge 💻 (~700 words)

Learning Objectives:

- Apply all learned concepts in integrated challenges
- Solve multi-step problems involving types and operators
- Debug type-related issues in existing code
- Build confidence in working with TypeScript fundamentals

Content Outline:

- Challenge 1: Temperature converter (arithmetic, conversion)
- Challenge 2: Age validator (comparison, logical operators)
- Challenge 3: Discount calculator (complex conditions, arithmetic)
- Challenge 4: Type fixer (identify and fix type issues)
- Challenge 5: Data processor (typeof, conversion, validation)
- Tips for approaching each challenge
- Common mistakes to avoid

Transitions: You've completed hands-on challenges covering all course topics. The final lesson assesses your understanding through a comprehensive knowledge check.

Key Principles:

- Break complex problems into smaller steps
- Test frequently with console.log()
- Handle edge cases and invalid inputs
- Write clean, readable code with semantic names

Hands-On Exercise - Challenge 1: Temperature Converter Create a temperature converter that:

- Accepts temperature value (number or string)
- Accepts unit ("C" or "F")
- Converts between Celsius and Fahrenheit
- Returns result with proper type
- Handles invalid inputs

Hands-On Exercise - Challenge 2: Age Validator Build an age validator that:

- Accepts age (various input types)
- Checks if age is valid (number, 0-150)
- Determines category: child (0-12), teen (13-17), adult (18-64), senior (65+)
- Returns object with validation status and category
- Uses proper comparison and logical operators

Hands-On Exercise - Challenge 3: Discount Calculator Create a discount system that:

- Accepts purchase amount, is member (boolean), is birthday (boolean)
- Applies 20% if member AND (amount > 100 OR birthday)
- Applies 10% if not member but amount > 200

- Applies 5% if none of above but amount > 50
- Calculates final price with discount
- Returns object with original, discount, and final price

Hands-On Exercise - Challenge 4: Type Fixer Given buggy code with type issues:

- Identify all type-related problems
- Fix loose equality (==) to strict (===)
- Fix implicit coercion issues
- Add proper type conversions
- Verify fixes with test cases

Hands-On Exercise - Challenge 5: Data Processor Build a data processor that:

- Accepts unknown input data
- Identifies input type using typeof
- Converts to appropriate type if needed
- Validates converted data
- Returns processed result or error message
- Handles all primitive types

Success Criteria:

- All challenges completed successfully
- Code handles edge cases and invalid inputs
- Proper types used throughout
- Strict equality used consistently
- Code is clean and well-organized
- console.log() used effectively for testing

05.1 Knowledge Check: Types and Operators 🧠 (~750 words)

Learning Objectives:

- Demonstrate comprehensive understanding of primitive data types
- Show mastery of arithmetic, assignment, comparison, and logical operators
- Apply operator precedence correctly
- Identify and fix common type-related mistakes

Assessment Format: 7 multiple-choice questions covering all course topics

Question 1: Data Types (Basic) What is the type of the following value: `"42"`?

- A) number
- B) string ✓
- C) boolean
- D) undefined

Explanation: Values surrounded by quotes are strings, even if they look like numbers. Use Number("42") to convert.

Question 2: Operators (Application) What is the result of: `10 + 5 * 2`?

- A) 30
- B) 20 ✓
- C) 12
- D) 25

Explanation: Multiplication happens before addition due to operator precedence: 5 * 2 = 10, then 10 + 10 = 20.

Question 3: Comparison Operators (Critical Thinking) What is the result of: `5 === "5"`?

- A) true
- B) false ✓
- C) undefined
- D) Error

Explanation: Strict equality (===) checks both value and type. 5 (number) and "5" (string) are different types, so result is false.

Question 4: Logical Operators (Application) Given: `const age = 25; const hasLicense = true;` What does `age >= 18 && hasLicense` evaluate to?

- A) true ✓
- B) false
- C) 25
- D) undefined

Explanation: AND (&&) requires both conditions to be true. age >= 18 is true AND hasLicense is true, so result is true.

Question 5: Type Identification (Analysis) What does `typeof null` return?

- A) "null"
- B) "undefined"
- C) "object" ✓
- D) null

Explanation: This is a JavaScript quirk. typeof null returns "object" due to historical reasons, not because null is actually an object.

Question 6: Type Conversion (Problem Solving) What is the best way to convert the string "123" to a number?

- A) +"123" or Number("123") ✓
- B) "123" - 0
- C) parseInt("123")
- D) All of the above work, but A is most explicit

Explanation: While all work, Number() is the clearest, most explicit method. The unary + also works but is less obvious.

Question 7: Complex Operators (Synthesis) What is wrong with this code?

```
 const age = 20;
if (age = 18) {
    console.log("Adult");
}
```

- A) Should use == instead of =
- B) Should use === instead of = ✓
- C) age should be let instead of const
- D) Nothing is wrong

Explanation: The code uses = (assignment) instead of === (comparison). This assigns 18 to age and always executes the if block.

Evaluation Criteria:

- **7/7 correct (100%):** Excellent! Complete mastery of types and operators
- **5-6 correct (71-86%):** Good understanding, review missed topics
- **3-4 correct (43-57%):** Adequate, but significant review needed

- **0-2 correct (0-29%):** Review entire course before continuing

Score Interpretation: Students scoring below 5/7 should review relevant sections before proceeding to control flow topics. Focus on:

- Distinction between primitive types
- Strict vs loose equality
- Logical operator truth tables
- Operator precedence rules
- Type conversion methods

Section 06: Conclusion

06.0 Your Foundation in TypeScript Data (~450 words)

Content Outline:

What You've Accomplished: In this course, you've built a solid foundation in TypeScript's data fundamentals. You now understand the six primitive data types—number, boolean, string, symbol, null, and undefined—and can identify them both visually and programmatically using typeof. You've mastered the four categories of operators: arithmetic for calculations, assignment for storing and updating values, comparison for making decisions, and logical for combining conditions. Most importantly, you've learned to write clean, type-safe TypeScript code following best practices like strict equality, semantic naming, and const-first declarations.

How This Connects to Your Journey: These fundamentals form the foundation for everything you'll build in TypeScript. Every variable you create, every calculation you perform, and every decision your programs make relies on the concepts from this course. The operators you've learned will appear in every conditional statement, loop, and function you write. Understanding types deeply will help you catch errors early and write more robust code as you progress through more advanced topics.

Next Steps: With data types and operators mastered, you're ready to combine these tools into powerful control flow structures. The next course will teach you to use conditional statements (if/else/switch) to make decisions and loops (for/while) to repeat operations. You'll apply the comparison and logical operators you learned here to control program flow, and you'll use arithmetic and assignment operators within your loops to solve real problems.

Resources for Continued Learning:

- TypeScript Handbook: Official documentation on types and operators
- MDN Web Docs: Comprehensive JavaScript/TypeScript operator reference
- 4Geeks Academy Resources: Additional exercises and examples
- Practice Platforms: Continue with Beginner TS exercises to reinforce concepts

Keep Building: Remember that programming is learned by doing, not by memorizing. The patterns you've practiced here—lowerCamelCase naming, const-first declarations, strict equality, semantic variable names—will become second nature with continued practice. Keep experimenting, keep testing with console.log(), and always ask "what type is this value?" when debugging. You're building strong foundations that will serve you throughout your entire coding journey.

Final Encouragement: You've completed a challenging but essential course. Understanding data types and operators might seem basic, but these are the atoms from which all programs are built. With this foundation solid, you're ready to construct increasingly complex and powerful TypeScript applications. Keep practicing, stay curious, and trust the process. Every expert developer started exactly where you are now. Keep coding!
